

Vasic-Digital-Reusable-Module-Suite

Создай один раз — используй везде: флот небольших, развязанных, независимо протестированных модулей Go и KMP.

Краткое описание

Большое семейство универсальных, переиспользуемых модулей, опубликованных под пространствами имён `digital.vasic.*` (Go) и Kotlin Multiplatform. Каждый модуль — самостоятельный, независимо протестированный и версионированный, и подключается как равноправный подмодуль к более крупным продуктам (Catalogizer, HelixAgent и всему флоту). Эта страница объединяет множество небольших утилит, которые терялись бы в общем потоке, будь у каждой своя отдельная страница.

Краткая характеристика

Тщательно отобранный набор развязанных модулей `digital.vasic.*` — инфраструктурные примитивы (аутентификация, кэш, база данных, конфигурация, наблюдаемость), строительные блоки для AI/агентов (RAG, VectorDB, эмбединги, MCP, агентность, планирование) и защитные механизмы LLM (RedTeam, Normalize) — а также зеркальный набор Kotlin Multiplatform. Каждый модуль универсален, протестирован и пригоден для повторного использования.

Подробное описание

Организация vasic-digital строится на одной структурной ставке: философии «конституция + множество развязанных переиспользуемых подмодулей», где универсальная функциональность никогда не пишется дважды. Вместо монолитов каждая переиспользуемая задача выносится в

отдельный небольшой модуль — со своим репозиторием, тестами и документацией — и жёстко изолируется, чтобы специфика потребителей никогда не просачивалась внутрь. Эта страница объединяет их, потому что по отдельности каждый модуль — это библиотека, которая терялась бы на фоне других страниц продуктов. Вместе же они становятся настоящим мультипликатором силы организации: частным инженерным активом, превращающим «создание нового продукта» в «сборку из проверенных компонентов», и конкретным подтверждением того, что этот флот не изобретает велосипед заново — он поддерживает один очень хороший велосипед и катит его повсюду.

Набор охватывает три кластера. **Инфраструктурные примитивы** (Go) обеспечивают базовую инфраструктуру, необходимую каждому сервису: auth (JWT/bcrypt), cache (Redis/TTL), database (миграции, двойная поддержка SQLite/PostgreSQL), config, middleware, observability (Prometheus/OpenTelemetry), ratelimiter, security, storage (S3/MinIO), streaming (концентратор WebSocket), eventbus, filesystem (мультипротокольная), discovery/mdns, http3, recovery, concurrency, lazy и другие. **Строительные блоки для AI/агентов** (Go) предоставляют основу для систем AI: rag, vectordb, embeddings, memory, conversation (сжатие контекста с бесконечной глубиной, событийный источник), mcp (Model Context Protocol), toolschema, skillregistry, agentic (оркестрация рабочих процессов на основе графов), planning (HiPlan/MCTS/Tree-of-Thoughts), benchmark (SWE-bench/HumanEval/MMLU), llmops, selfimprove (моделирование вознаграждения/RLHF) и toon (Token-Oriented Object Notation). **Защитные механизмы LLM** обеспечивают инструментарий для устойчивости к атакам: RedTeam (YAML-ориентированные атакующие сценарии), Normalize (канонизация враждебных входных данных). Параллельный набор **Kotlin Multiplatform** дублирует ключевые модули (Auth-KMP, Database-KMP, Security-KMP, UI-Components-KMP и др.) для кроссплатформенных приложений.

Почему мы это создали

Каждый раз разрабатывать множество продуктов (Catalogizer, HelixAgent, Herald и другие) с нуля — расточительно и непоследовательно. Выделение всех типовых задач в независимые, протестированные модули позволяет распространять исправления и улучшения на всю экосистему, а каждый новый продукт собирается из проверенных компонентов.

Почему это меняет правила игры

По сути, это частная «стандартная библиотека» для создания бэкендов, ориентированных на AI, — тот самый слой, который большинство команд никогда не успевают построить, потому что слишком заняты повторным решением задач аутентификации, кэширования и инфраструктурной «сантехники» RAG в пятый раз. Здесь инфраструктурные примитивы, строительные блоки AI и защитные механизмы LLM существуют в виде готовых, независимо протестированных модулей. Именно это позволяет небольшой команде выпускать системы промышленного уровня в темпе, который обычно требует гораздо больших ресурсов, — и делать это без накопления долга дублирования, неизбежного в других случаях.

Что здесь инновационного

- Дисциплина децентрализации на уровне всей экосистемы (CONST-051): субмодули рассматриваются как равноправные кодовые базы, не содержащие специфики потребителей.
- Выделенный слой примитивов AI (RAG, VectorDB, эмбединги, MCP, ToolSchema, агентные системы, планирование, LLMOps) в виде переиспользуемых модулей.
- Кластер защитных механизмов LLM (RedTeam, Normalize) для обеспечения устойчивости к атакам.
- Параллельные наборы модулей Go и Kotlin Multiplatform, использующие единые соглашения.

Проблемы и решения

- **Избегание спагетти-зависимостей между десятками модулей:** решено с помощью контракта децентрализации в конституции и инъекции специфики потребителей на уровне рантайма.
- **Поддержание согласованности и тестируемости множества модулей:** решено за счёт единых соглашений (тесты/документация/задачи на каждый модуль) и управленческого каркаса HelixConstitution.
- **Кросс-платформенный охват:** решено с помощью зеркалирования ключевых модулей в Kotlin Multiplatform.

Технологический стек (зачем и как)

- **Go** — основная часть модулей (digital.vasic.*).

- **Kotlin Multiplatform** — кросс-платформенные зеркальные модули (Auth/Database/Security/UI/Concurrency/RateLimiter-KMP).
- **Redis / PostgreSQL / SQLite** — примитивы кэша, баз данных и хранилищ.
- **Prometheus / OpenTelemetry** — модуль наблюдаемости.
- **WebSocket / HTTP/3 (quic-go) / mDNS** — сетевые модули.
- **Vector DB / embeddings / RAG / MCP** — модули примитивов AI.
- **YAML** — RedTeam-фикстуры и конфигурации для тестирования на устойчивость к атакам.

НЕПРОВЕРЕНО / В РАЗРАБОТКЕ: несколько репозиторий организации помечены как «SCAFFOLD / WIP» (например, PliniusCommon, I-LLM, HyperTune, AutoTemp, Veritas, Ouroboros, Claritas, LeakHub, GandalfSolutions). Представляйте их как ранние/черновые версии, а не готовые к релизу.